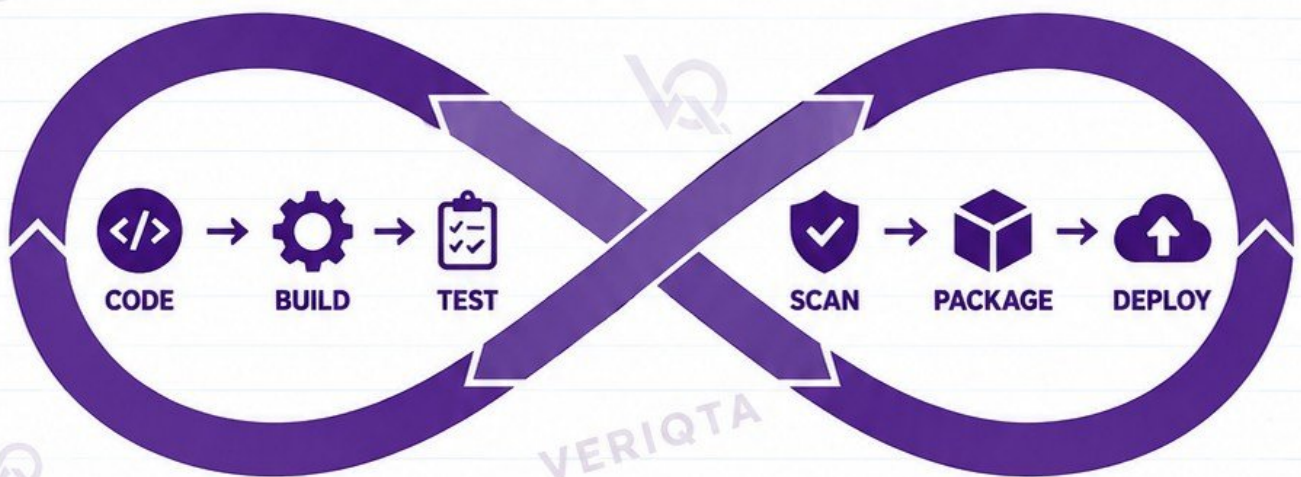


CI/CD INTERVIEW SCENARIOS

20 REAL-WORLD CI/CD SITUATIONS.
TROUBLESHOOT. EXPLAIN. SOLVE.



SCENARIO
BASED
LEARNING



REAL-WORLD
PROBLEMS



INTERVIEW
FOCUSED



PRODUCTION
RELEVANT



SOLUTION
DRIVEN



PRACTICE THINKING LIKE A DEVOPS ENGINEER.
EXPLAIN YOUR APPROACH. SOLVE WITH CONFIDENCE.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

CI/CD PIPELINE FAILS IMMEDIATELY AFTER A COMMIT



DEVELOPER PUSHES CODE AND THE PIPELINE FAILS



1. WHERE DO YOU START?

- ✓ Don't panic. Take a structured approach.
- ✓ Go to the pipeline run for the failed commit.
- ✓ Understand what stage failed and why.
- ✓ Avoid rerunning the pipeline immediately.
- ✓ Focus on facts, not guesses.

2. IDENTIFY THE FAILED STAGE



Pinpoint the first failing stage.
All stages after it may also fail as a result.



3. READ LOGS BEFORE RERUNNING

- ✓ Open the logs for the failed stage.
- ✓ Read the error message carefully.
- ✓ Look for the first error, not the last.
- ✓ Understand what the job was trying to do.
- ✓ Collect the exact error and stack trace.



4. COMPARE WITH THE LAST SUCCESSFUL PIPELINE

WHAT TO COMPARE

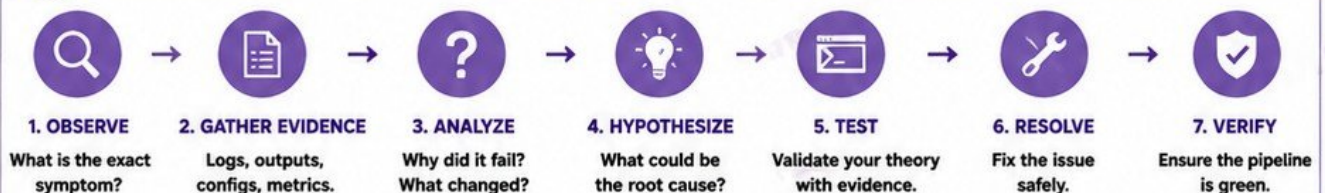
- ✓ Code changes
- ✓ Dependencies / versions
- ✓ Configuration changes
- ✓ Environment variables
- ✓ Secrets / tokens
- ✓ Infrastructure changes
- ✓ External services

HOW TO COMPARE

- 👁️ Compare pipeline runs
- 🕒 Check timestamps
- </> Review code diff
- ☰ Check recent changes
- 👤 Ask: What changed?



5. INTERVIEW FOCUS: STRUCTURED TROUBLESHOOTING



QUICK CHECKLIST

- ✓ Identify the failed stage
- ✓ Read logs carefully
- ✓ Compare with last successful run
- ✓ Look for what changed
- ✓ Form a theory based on evidence
- ✓ Fix and verify



REMEMBER

GOOD ENGINEERS DON'T GUESS.
THEY READ, COMPARE, THINK, AND SOLVE.



STRUCTURE. EVIDENCE. PATIENCE. THAT'S HOW YOU SOLVE PIPELINE FAILURES LIKE A PRO.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

BUILD WORKS LOCALLY BUT FAILS IN CI

DEVELOPER SAYS: "IT WORKS ON MY MACHINE"

1. RUNTIME AND TOOL VERSIONS



LOCAL MACHINE



CI ENVIRONMENT

- ✓ Compare language/runtime versions.
- ✓ Compare build tools and package managers.
- ✓ Check system dependencies and OS versions.
- ✓ Use version files: .nvmrc, .tool-versions, Dockerfile.
- ✓ Print versions in CI logs for full visibility.



Mismatch in versions is a top cause of "works on my machine" issues.

2. MISSING ENVIRONMENT VARIABLES



- ✓ Check for missing environment variables in CI.
- ✓ Compare local .env files with CI configuration.
- ✓ Look for secrets, API keys, tokens, and database URLs.
- ✓ Fail fast with clear error messages when variables are missing.



Applications often behave differently without required configuration.

3. FILE PATHS AND CASE SENSITIVITY



MyFile.ts



myfile.ts

- ✓ Local machines (Windows/macOS) may ignore case.
- ✓ CI (Linux) is case-sensitive.
- ✓ Check file names, imports, and paths.
- ✓ Avoid hardcoded absolute paths.
- ✓ Use relative paths when possible.



Case sensitivity and path differences cause many CI build failures.

4. DEPENDENCIES AND CLEAN BUILD ENVIRONMENTS



CI uses clean environments. Nothing is cached.

- ✓ Missing or undeclared dependencies will break the build.
- ✓ Check package.json, requirements.txt, pom.xml, etc.
- ✓ Ensure lock files are committed (package-lock.json, pnpm-lock.yaml, Pipfile.lock, etc.).
- ✓ Avoid relying on globally installed tools.



If it's not declared, CI won't have it. Always declare your dependencies.

5. WHAT TO DO WHEN IT FAILS IN CI

- 1 Read the full error and stack trace.
- 2 Identify the exact failing step.
- 3 Compare with the last successful build.
- 4 Check versions, configs, and environment variables.
- 5 Reproduce locally using the CI environment (Docker or exact versions).
- 6 Fix the root cause, not the symptom.
- 7 Commit, push, and verify.



Understand the environment. Reproduce the issue. Fix the root cause.

6. INTERVIEW FOCUS: ENVIRONMENT CONSISTENCY



UNDERSTAND THE DIFFERENCES
Know why local and CI environments differ.



GATHER EVIDENCE
Collect logs, configs, and version details.



FORM HYPOTHESES
Identify what could cause the failure.



TEST SYSTEMATICALLY
Check one thing at a time.



ENSURE CONSISTENCY
Make environments predictable and repeatable.



CONSISTENCY IS THE FOUNDATION OF RELIABLE CI/CD PIPELINES.
REPRODUCIBLE ENVIRONMENTS = PREDICTABLE BUILDS.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

PIPELINE SUDDENLY FAILS WITHOUT A CODE CHANGE

⚠ SAME COMMIT PREVIOUSLY WORKED, BUT NOW IT FAILS



WHAT THIS MEANS

- ✓ The code did not change.
- ✓ The commit worked before.
- ✓ The failure is caused by something outside the repository.
- ✓ Your job is to find out what changed in the environment or external systems.

COMMON REASONS



EXPIRED CREDENTIALS



DEPENDENCY OR REGISTRY CHANGES



RUNNER CHANGES



EXTERNAL SERVICE OUTAGES

INVESTIGATION CHECKLIST



1. CHECK CREDENTIALS

Expired tokens or passwords cause authentication failures.

- Look for 401 or 403 errors in logs.
- Verify secrets, tokens, and service account keys.
- Check expiration dates and rotate if needed.



2. CHECK DEPENDENCIES AND REGISTRIES

Packages or container images may have changed.

- Confirm package versions and lock files.
- Check if a dependency was yanked or updated.
- Verify registry availability and permissions.



3. CHECK RUNNER CHANGES

CI runner updates or configuration changes can break builds.

- Runner version changes?
- New configuration, labels, or environment variables?
- Check runner logs and disk/memory usage.



4. CHECK EXTERNAL SERVICES

Third-party services may be down or degraded.

- Check status pages (e.g., AWS, GitHub, Docker Hub).
- Look for network errors, timeouts, or DNS issues.
- Validate API, webhook, and external integrations.



5. CHECK TIMELINE

Build a timeline to identify what changed.

- When did the last successful pipeline run?
- When did the failure start?
- What changed around that time?



6. GATHER EVIDENCE

Collect logs, error messages, and environment details.

- Read full logs carefully before rerunning.
- Note error messages and timestamps.
- Capture screenshots or log snippets.

INVESTIGATION WORKFLOW



1. OBSERVE
Understand the failure.



2. IDENTIFY
What changed outside the repo?



3. GATHER EVIDENCE
Logs, configs, environment.



4. HYPOTHESIZE
Form a theory based on facts.



5. TEST & FIX
Resolve the issue and verify.

QUICK TIPS

- ✓ Never assume the code is the problem.
- ✓ Check the environment first.
- ✓ Read logs before rerunning.
- ✓ Document what you find.
- ✓ Communicate clearly with the team.
- ✓ Prevent recurrence.



**GREAT ENGINEERS INVESTIGATE WHAT CHANGED.
NOT JUST THE CODE. THINK BEYOND THE REPOSITORY.**



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

DEPENDENCY INSTALLATION STARTS FAILING

⚠ PACKAGE CANNOT BE DOWNLOADED

1. REGISTRY AVAILABILITY



- ✓ Is the package registry online?
- ✓ Check status page and uptime.
- ✓ Look for maintenance windows.
- ✓ Verify registry endpoint URL.

2. AUTHENTICATION



- ✓ Are credentials valid?
- ✓ Check tokens, API keys, or passwords.
- ✓ Verify permissions/roles.
- ✓ Check for expired tokens.

3. NETWORK / DNS PROBLEMS



- ✓ Can the CI runner reach the registry?
- ✓ Check DNS resolution.
- ✓ Look for firewall or proxy restrictions.
- ✓ Test connectivity (ping, curl).

4. VERSION CHANGES



- ✓ Did the package version change?
- ✓ Is the requested version deprecated or removed?
- ✓ Check release notes.
- ✓ Verify version constraints.

5. LOCK FILES



- ✓ Is a lock file present?
- ✓ Lock files ensure the same versions every time.
- ✓ Check if the lock file is outdated or corrupted.
- ✓ Regenerate lock file only when necessary.

6. PINNED vs FLOATING



- ✓ Pinned (exact versions) = reproducible builds.
- ✓ Floating (latest, ^, ~) = unpredictable results.
- ✓ Use pinned versions for stability in CI/CD.

7. PACKAGE MANAGER CACHE / STATE



- ✓ Corrupted or stale cache can break installs.
- ✓ Clear cache and retry.
- ✓ Ensure CI uses clean environments.
- ✓ Avoid relying on globally installed packages.

8. TRANSITIVE DEPENDENCIES



- ✓ A dependency of a dependency may be the issue.
- ✓ Check updated transitive dependencies.
- ✓ Use tools to inspect dependency trees.

INVESTIGATION CHECKLIST



READ THE FULL ERROR
Identify the exact package and error message.



CHECK REGISTRY STATUS
Confirm availability and any ongoing incidents.



VERIFY AUTHENTICATION
Test credentials and permissions.



TEST NETWORK CONNECTIVITY
Check DNS, firewall, proxy, and internet access.



REVIEW VERSION CONSTRAINTS
Ensure the requested version exists and is compatible.



CHECK LOCK FILES
Validate lock file integrity and versions.



CLEAR CACHE AND RETRY
Clean package manager cache and workspace.



REPRODUCE LOCALLY
Reproduce the failure in a clean local environment or Docker.



DOCUMENT AND FIX
Fix the root cause, update configs, and commit the change.

COMMON ERROR EXAMPLES



- npm ERR! code E404 - Not found
- pip ERROR: Could not find a version that satisfies the requirement
- 403 Forbidden - Authentication failed
- ENOTFOUND / EAI_AGAIN - DNS resolution failed
- Network timeout while downloading package
- Version conflict - Unable to resolve dependency tree



INTERVIEW FOCUS: REPRODUCIBLE BUILDS

- ✓ Reproducible builds are predictable and reliable.
- ✓ Pin your dependencies.
- ✓ Use lock files.
- ✓ Avoid relying on floating versions.
- ✓ Keep environments consistent.
- ✓ Document and automate everything.



REPRODUCIBLE BUILDS SAVE TIME, PREVENT SURPRISES, AND BUILD TRUST IN YOUR CI/CD PIPELINES.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

CI/CD SECRET OR TOKEN HAS EXPIRED



PIPELINE SUDDENLY GETS AUTHENTICATION ERRORS

1. RECOGNIZING AUTH ERRORS

401

UNAUTHORIZED

- Invalid or missing credentials.
- Authentication failed.

403

FORBIDDEN

- Credentials are valid, but access is denied.
- Insufficient permissions or expired token.



Always check the HTTP status code and error message in the logs.

2. SECRET STORES & CI VARIABLES


CI/CD
VARIABLES

SECRET
STORES

CLOUD
SECRETS

- Secrets are stored securely.
- Access is granted to the pipeline.
- Environment variables are correctly set.
- Secrets are masked in logs.
- Use least privilege access.



Never hardcode secrets in code or configuration files.

3. TOKEN EXPIRATION



- Tokens and keys can expire.
- Check expiration date and TTL.
- Expired tokens cause 401 or 403 errors.
- Short-lived tokens improve security but require rotation.



Monitor token lifetimes and set alerts for upcoming expirations.

4. TOKEN ROTATION



- Rotate tokens before they expire.
- Use automation for rotation.
- Update CI/CD variables or secret stores.
- Test after rotation to confirm access.
- Revoke old tokens immediately.



Automate rotation. Manual rotation should be a last resort.

5. NEVER EXPOSE SECRETS IN LOGS



- Secrets in logs can be read by anyone.
- Mask secrets in CI/CD settings.
- Avoid echoing secrets in scripts.
- Use secret masking and redaction.
- Review logs for accidental exposure.



If a secret is exposed, rotate it immediately.

6. QUICK CHECKLIST



- Look for 401 or 403 errors.
- Check token expiration.
- Verify CI/CD variables are set.
- Confirm permissions and scopes.
- Check secret store access.
- Rotate token if expired.
- Rerun the pipeline after fixing.
- Document the resolution.

7. SECURE CREDENTIAL TROUBLESHOOTING WORKFLOW



1. OBSERVE
Read the error message and status code.



2. IDENTIFY
Confirm it's an authentication issue.



3. VERIFY
Check token, permissions, and expiration.



4. CHECK ACCESS
Verify secret store and CI/CD variable availability.



5. ROTATE / FIX
Rotate or update the credential as needed.



6. TEST
Rerun the pipeline and confirm success.



7. PREVENT
Set alerts and improve rotation process.



COMMON CAUSES

- Expired token or key
- Revoked or rotated credential not updated in CI
- Missing or incorrect CI/CD variable
- Insufficient permissions or wrong scope
- Service account disabled or deleted



INTERVIEW FOCUS

SECURE CREDENTIAL TROUBLESHOOTING IS ABOUT CONFIDENTLY IDENTIFYING AUTHENTICATION ISSUES, FIXING THEM SAFELY, AND PREVENTING RECURRENCE.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

CI RUNNER IS OFFLINE OR UNAVAILABLE

⚠ JOBS REMAIN QUEUED

1. WHAT YOU SEE



- ✓ Jobs are stuck in "Queued".
- ✓ No runners available for this job.
- ✓ Pipeline does not start or remains pending.



The issue is usually on the runner side, not the pipeline configuration.

2. POSSIBLE CAUSES



Runner is offline or not connected.



Runner service has stopped.



Insufficient resources (CPU, Memory, Disk).



Runner labels/tags do not match the job requirements.



Network connectivity issues.

3. RUNNER REGISTRATION



- ✓ Verify the runner is registered in the CI/CD platform.
- ✓ Check registration token validity.
- ✓ Re-register the runner if it is missing.
- ✓ Ensure the runner is assigned to the correct project/group.

4. RUNNER SERVICE STATUS



- ✓ Check the runner service is running.
- ✓ Restart the service if it has stopped.
- ✓ Check for crashes in service logs.
- ✓ Enable auto-restart on boot.
- ✓ Verify runner version compatibility.

5. CHECK RESOURCES



CPU



MEMORY



DISK

- ✓ High CPU usage can stall jobs.
- ✓ Low memory can cause job failures or runner crashes.
- ✓ Disk full prevents new jobs and artifact storage.



Keep at least 20% free disk space and monitor resource usage.

6. CHECK LABELS / TAGS



RUNNER LABELS



JOB REQUIREMENTS

- ✓ Job will remain queued if no runner matches the required labels.
- ✓ Verify labels assigned to the runner.
- ✓ Verify labels required by the job.
- ✓ Use generic labels only if safe.



Labels must match exactly (case-sensitive).

7. CHECK NETWORK CONNECTIVITY



- ✓ Runner must reach the CI/CD server.
- ✓ Check firewall rules and proxy settings.
- ✓ Test outbound internet access.
- ✓ Check DNS resolution.
- ✓ Verify ports (80/443) are open.



Network issues are a common cause of offline runners.

8. TROUBLESHOOTING WORKFLOW



OBSERVE
Jobs are queued.



CHECK RUNNER STATUS
Is the runner online and connected?



CHECK SERVICE
Is the runner service running?



CHECK RESOURCES
CPU, memory, and disk OK?



CHECK LABELS
Do labels match the job?



CHECK NETWORK
Can the runner reach the server?



RESOLVE & VERIFY
Fix the issue and run a test job.

9. COMMON COMMANDS



```
# Check runner service status
sudo systemctl status gitlab-runner
# Restart runner service
sudo systemctl restart gitlab-runner
# View runner logs
sudo journalctl -u gitlab-runner -f
# Check disk space
df -h
# Check memory usage
free -h
# Test connectivity
curl -I https://gitlab.com
```



10. INTERVIEW FOCUS

- ✓ Separate pipeline issues from infrastructure issues.
- ✓ Follow a logical troubleshooting flow.
- ✓ Collect evidence before making changes.
- ✓ Understand runner lifecycle and health.
- ✓ Communicate clearly with the team.



A healthy runner = a reliable pipeline.
Always confirm the runner before blaming the pipeline.

QUICK CHECKLIST



- ✓ Runner is online and registered.
- ✓ Runner service is running.
- ✓ Sufficient CPU, memory, and disk.
- ✓ Labels match the job.
- ✓ Network connectivity is working.
- ✓ Runner version is compatible.
- ✓ Test job runs successfully.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

PIPELINE RUNS OUT OF DISK SPACE



BUILDS BEGIN FAILING UNEXPECTEDLY

1. COMMON CAUSES



WORKSPACE GROWTH

Large repositories, many files, or repeated checkouts.



DOCKER IMAGES & BUILD CACHE

Dangling images, layers, and build cache consume space.



ARTIFACTS

Large artifacts or old artifacts not being cleaned up.



LOGS

Verbose logs, old job logs, and system logs grow over time.

2. SYMPTOMS



- Build fails with "No space left on device"
- Docker build or pull fails
- Artifacts cannot be saved
- Runner becomes slow or unresponsive
- Jobs fail at random steps



IMPACT

Pipelines fail, deployments stop, and developers lose trust in CI.

3. RUNNER DISK INVESTIGATION

	WHAT TO CHECK	COMMAND EXAMPLES
	Check overall disk usage	<code>df -h</code>
	Find largest directories	<code>sudo du -sh /* 2>/dev/null sort -hr head -n 20</code>
	Check workspace size	<code>du -sh \$CI_PROJECT_DIR</code>
	Check Docker disk usage	<code>docker system df</code>
	List large Docker images	<code>docker images --format "table {{.Repository}}\t{{.Tag}}\t{{.Size}}" sort -k3 -hr</code>
	Check large containers	<code>docker ps -a --size</code>
	Check system logs size	<code>sudo du -sh /var/log</code>
	Find large files	<code>sudo find / -type f -size +500M -exec ls -lh {} \;</code>

4. SAFE CLEANUP ACTIONS

- Remove unused Docker images `docker image prune -af`
- Remove unused containers `docker container prune -f`
- Remove unused volumes `docker volume prune -f`
- Remove builder cache (if safe) `docker builder prune -af`
- Clean CI workspace between jobs `rm -rf $CI_PROJECT_DIR/*`
- Expire old artifacts Set retention policies
- Rotate or limit logs Configure log rotation

5. PREVENTION BEST PRACTICES

- Use smaller base images
- Clean workspace after each job
- Cache dependencies efficiently
- Limit artifact size and retention period
- Prune Docker resources regularly on runners
- Monitor disk usage and set alerts
- Use ephemeral runners when possible



INTERVIEW FOCUS

CI INFRASTRUCTURE TROUBLESHOOTING
REAL ENGINEERS FIND THE ROOT CAUSE,
CLEAN SAFELY, AND PREVENT RECURRENCE.



REMEMBER

BLINDLY DELETING THINGS CAN BREAK BUILDS.
CLEAN WITH CARE. UNDERSTAND BEFORE YOU REMOVE.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

AUTOMATED TESTS FAIL DURING THE PIPELINE



BUILD SUCCEEDS BUT TEST STAGE FAILS

TYPICAL CI/CD PIPELINE



PIPELINE STOPS HERE

The test stage failed.
No package is produced.
No deployment happens.

1. TYPES OF TEST FAILURES



UNIT TEST FAILURES

Failures in small, isolated functions or components.



INTEGRATION TEST FAILURES

Failures when components or services work together.



E2E / SYSTEM TEST FAILURES

Failures in full flow or end-to-end user scenarios.

2. COMMON CAUSES

- ✗ New code introduced a bug.
- ✗ Flaky or unstable tests.
- ✗ Test data issues or unexpected state.
- ✗ Missing or unmocked dependencies.
- ✗ External services unavailable.
- ✗ Environment differences (OS, versions, locale, configs).
- ✗ Time-sensitive tests.
- ✗ Resource limits (CPU, memory, DB connections).
- ✗ Race conditions in tests.

3. HOW TO TROUBLESHOOT TEST FAILURES

STEP	WHAT TO DO	WHY IT MATTERS
 REVIEW THE FAILED TESTS	Open the test report and logs.	Identify exactly which tests failed and the error message.
 CHECK THE FAILURE PATTERN	Is it one test or many? Same failure or different?	Helps determine if it's a code issue or environment/dependency issue.
 COMPARE WITH LAST SUCCESSFUL RUN	Check what changed since the last green build.	Find recent code, dependency, or config changes.
 VERIFY TEST DEPENDENCIES	Databases, APIs, services, queues, files, feature flags.	External test dependencies are a common source of failures.
 REPRODUCE LOCALLY	Run the same tests locally in a clean environment if possible.	Confirms if the issue is code-related or environment-related.
 CHECK ENVIRONMENT DIFFERENCES	OS, runtime, libraries, env vars, timezone, locale, network.	"It works on my machine" is usually an environment mismatch.
 RERUN CAREFULLY IF NEEDED	Rerun only after understanding the failure.	Avoid wasting time and hiding real problems.

4. WHY BYPASSING TESTS IS DANGEROUS

⚠️ NEVER SKIP OR IGNORE FAILING TESTS IN CI

- ✗ Bugs go to production.
- ✗ Security vulnerabilities may be introduced.
- ✗ Customer trust and system stability are at risk.
- ✗ Debugging in production is more expensive.
- ✗ It breaks the quality culture and feedback loop.
- ✗ It delays finding the real root cause.



5. INTERVIEW FOCUS: QUALITY GATES

- ✓ How do you ensure only good code is released?
- ✓ How do you handle flaky tests?
- ✓ How do you manage test data and environments?
- ✓ When is it acceptable to retry tests?
- ✓ How do you prevent tests from slowing down delivery?
- ✓ How do you balance quality and speed?



REMEMBER

TESTS ARE YOUR SAFETY NET.

A STRONG PIPELINE STOPS BAD CODE BEFORE IT REACHES USERS.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

SECURITY SCAN BLOCKS THE PIPELINE

! APPLICATION BUILDS SUCCESSFULLY BUT CANNOT PROCEED

TYPICAL CI/CD PIPELINE



! PIPELINE STOPS HERE

Security issues detected.
Policy prevents release.
Risk must be reviewed.

1. WHY SCANS BLOCK THE PIPELINE

- ✓ **VULNERABILITY SCANNING FINDS RISKS**
Scanners detect known CVEs in code, dependencies, and container images.
- ✓ **SEVERITY THRESHOLDS ARE ENFORCED**
High or Critical issues block the pipeline based on policy.
- ✓ **SECURITY IS A QUALITY GATE**
Untrusted code should never reach production.
- ✓ **RISK MUST BE ADDRESSED**
Fix, justify, or accept with documented approval.



2. COMMON REASONS FOR SCAN FAILURES

- ✗ High or Critical vulnerabilities found.
- ✗ Dependency vulnerabilities (direct or transitive).
- ✗ Outdated base images with known CVEs.
- ✗ Secrets detected in code or configuration.
- ✗ Misconfiguration (Dockerfile, IaC, manifests).
- ✗ Policy violations (licenses, hardcoded secrets).
- ✗ Scanner cannot authenticate or access data.

3. KEY CONCEPTS ENGINEERS MUST UNDERSTAND

-  **VULNERABILITY SCANNING**
Automated detection of known security issues.
-  **SEVERITY THRESHOLDS**
Defined levels (e.g., Critical, High) that determine blocking behavior.
-  **FALSE POSITIVES**
Scanners can flag issues that are not real risks in your context.
-  **DEPENDENCY VULNERABILITIES**
Transitive dependencies can introduce risks you did not directly add.
-  **EXCEPTIONS & RISK ACCEPTANCE**
Documented approvals for acceptable risks with mitigation plans and expiry dates.

4. WHAT TO DO WHEN THE SCAN BLOCKS THE PIPELINE

- | | |
|---------------------------------|--|
| 1 REVIEW THE SCAN REPORT | Understand what was found and why it is considered a risk. |
| 2 CHECK SEVERITY & POLICY | Compare findings against severity thresholds and policy rules. |
| 3 VERIFY FALSE POSITIVES | Confirm if the issue is exploitable in your environment. |
| 4 FIX THE ISSUE | Update dependencies, base images, or code to remove the risk. |
| 5 RE-SCAN AND VERIFY | Ensure the issue is resolved and the pipeline can proceed. |
| 6 REQUEST EXCEPTION (IF NEEDED) | Provide justification, mitigation, owner, and expiry. |
| 7 DOCUMENT EVERYTHING | Keep audit logs of decisions and approvals. |

5. WHY YOU SHOULD NEVER DISABLE THE SCANNER

- ✗ It hides real vulnerabilities from reaching production.
- ✗ It increases the chance of security incidents and breaches.
- ✗ It violates security policies and compliance requirements.
- ✗ It destroys trust in the CI/CD process.
- ✗ Attackers target known unpatched vulnerabilities.
- ✗ Shortcuts today become major incidents tomorrow.



6. INTERVIEW FOCUS: DEVSECOPS DECISIONS

- ✓ How do you handle a Critical vulnerability in a dependency?
- ✓ How do you reduce false positives without lowering security?
- ✓ When is it acceptable to request an exception?
- ✓ What information must be included in an exception request?
- ✓ How do you ensure exceptions are reviewed and time-bound?
- ✓ How do you balance security, delivery speed, and business risk?
- ✓ How do you improve the security posture over time?



REMEMBER

**A GREEN BUILD MEANS NOTHING IF THE CODE IS NOT SAFE.
SECURITY IS EVERY ENGINEER'S RESPONSIBILITY.**



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

DOCKER IMAGE BUILD FAILS IN CI



DOCKER BUILD FAILS AND THE PIPELINE CANNOT CONTINUE

TYPICAL CI/CD PIPELINE



PIPELINE STOPS HERE

DOCKER image build failed.
Image not created.
Nothing is pushed.

1. COMMON REASONS FOR DOCKER BUILD FAILURES

- ✗ Dockerfile syntax or instruction errors.
- ✗ Missing or incorrect build context.
- ✗ COPY or ADD path is incorrect.
- ✗ Missing files in the build context (ignored by .dockerignore).
- ✗ Dependency installation failures (apt, pip, npm, etc).
- ✗ Network or DNS issues during build.
- ✗ Build arguments not provided or incorrect.
- ✗ Resource limits (disk space, memory) on the runner.
- ✗ Base image not found or access denied.
- ✗ Image size is too large or storage is full.



2. HOW TO TROUBLESHOOT DOCKER BUILD FAILURES

- | 1 | READ THE ERROR | Read the full error message at the bottom of the build logs. |
|----|------------------------|--|
| 2 | CHECK THE DOCKERFILE | Review the failing instruction and syntax. |
| 3 | VERIFY BUILD CONTEXT | Ensure all required files are in the context. |
| 4 | CHECK .DOCKERIGNORE | Make sure needed files are not excluded. |
| 5 | VERIFY COPY PATHS | Paths in COPY must exist relative to build context. |
| 6 | CHECK DEPENDENCIES | Verify package names, versions, and repositories. |
| 7 | CHECK NETWORK | Confirm registry and package sources are reachable. |
| 8 | CHECK BUILD ARGUMENTS | Ensure all ARG/--build-arg values are provided. |
| 9 | CHECK RUNNER RESOURCES | Ensure enough disk space and memory. |
| 10 | REPRODUCE LOCALLY | Run the same build locally to reproduce. |

3. DOCKER BUILD CHECKLIST

- | Check | Command |
|----------------------------------|----------------------------------|
| ✗ Check disk space | df -h |
| ✗ Check Docker system usage | docker system df |
| ✗ Prune unused data (safe) | docker system prune -af |
| ✗ Inspect build context size | du -sh . |
| ✗ List files in context | ls -la |
| ✗ Validate Dockerfile syntax | docker build --no-cache . |
| ✗ Use plain progress for details | DOCKER_BUILDKIT=0 docker build . |
| ✗ Build with no cache | docker build --no-cache . |
| ✗ Pass build args explicitly | --build-arg KEY=value |
| ✗ Check base image availability | docker pull <image>:<tag> |

4. COMMON DOCKERFILE MISTAKES

- ✗ Wrong or missing FROM image.
- ✗ COPY ./app /app but ./app does not exist in context.
- ✗ RUN apt-get update && apt-get install ... without cleanup (causes large images).
- ✗ Leaving unnecessary files in the image.
- ✗ Not using .dockerignore to reduce context size.
- ✗ Hardcoding secrets or tokens.
- ✗ Using latest tag instead of specific versions.



5. WHY IMAGE SIZE AND STORAGE MATTER

- ✓ Large images slow down builds and deployments.
- ✓ Consume more registry storage and bandwidth.
- ✓ Increase attack surface and patching effort.
- ✓ Can cause build failures when storage is full.
- ✓ Smaller images are faster, cheaper, and more secure.



6. INTERVIEW FOCUS: CONTAINER BUILD TROUBLESHOOTING

- ✓ How do you read and interpret Docker build logs?
- ✓ How do you debug COPY or context issues?
- ✓ How do you handle dependency installation failures?
- ✓ How do you reduce image size safely?
- ✓ How do you ensure reproducible and reliable builds?



REMEMBER

A BROKEN IMAGE STOPS DELIVERY.

READ THE LOGS, FIND THE ROOT CAUSE, AND FIX IT CORRECTLY.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

PIPELINE CANNOT PUSH AN IMAGE TO THE REGISTRY



IMAGE BUILDS SUCCESSFULLY BUT PUBLISHING FAILS

TYPICAL CI/CD PIPELINE



PIPELINE STOPS HERE

Image build succeeded but push to registry failed.
No image available for deployment.

1. COMMON REASONS PUSH FAILS

- Registry authentication failed (invalid/expired token).
- Insufficient permissions to push to the repository.
- Repository does not exist or is misspelled.
- Incorrect image name, tag, or registry URL.
- Network connectivity or DNS issues.
- Registry service is down or degraded.
- Rate limiting or quota limits reached.
- Large image upload interrupted or timed out.



2. HOW TO TROUBLESHOOT

1 CHECK THE ERROR MESSAGE	Read the full error in logs. It tells you what failed.
2 VERIFY REGISTRY CREDENTIALS	Check username, token, or service account.
3 CHECK PERMISSIONS	Ensure the user has push access to the repository.
4 VERIFY REPOSITORY EXISTS	Confirm the repository name and path are correct.
5 CHECK IMAGE NAME & TAG	Validate the full image name and tag format.
6 CHECK NETWORK & DNS	Ensure the runner can reach the registry.
7 CHECK REGISTRY HEALTH	Confirm the registry service is online and available.
8 CHECK QUOTAS & LIMITS	Verify storage quota and rate limits.
9 RETRY WITH VERBOSE MODE	Use --debug for more details when supported.

3. QUICK COMMANDS TO INVESTIGATE

Test registry connectivity	ping registry.example.com
Check DNS resolution	nslookup registry.example.com
Docker login (debug)	docker login -u user -p \$TOKEN --debug registry.example.com
List local images	docker images
Tag the image	docker tag app:1.0 \ registry.example.com/team/app:1.0
Push with debug	docker push --debug \ registry.example.com/team/app:1.0
Inspect disk space	df -h
Check Docker daemon	docker info
Check proxy settings	env grep -i proxy

4. COMMON ERROR EXAMPLES

- unauthorized: authentication required
→ Invalid or expired credentials.
- denied: requested access to the resource is denied → Insufficient permissions.
- repository does not exist or may require 'docker login' → Wrong repository name or path.
- dial tcp: lookup registry.example.com: no such host → DNS or network issue.
- i/o timeout → Network timeout or registry down.
- too many requests: rate limit exceeded → Rate limit or quota exceeded.

BEST PRACTICES FOR RELIABLE PUSH

- Use long-lived or properly rotated tokens.
- Grant minimal required permissions.
- Use consistent image naming and tagging (e.g., include commit SHA).
- Handle retries with backoff for transient failures.
- Monitor registry availability and storage usage.
- Keep runners in reliable networks with good bandwidth.



6. INTERVIEW FOCUS: ARTIFACT DELIVERY

- How do you securely authenticate to a registry?
- How do you verify you have push permissions?
- How do you handle registry downtime?
- How do you ensure correct image naming and tagging?
- What steps do you take when a push fails?
- How do you prevent this issue from recurring?



REMEMBER

A BUILD WITHOUT DELIVERY HAS NO VALUE.

ENSURE YOUR ARTIFACT REACHES THE REGISTRY SAFELY AND RELIABLY.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

WRONG DOCKER IMAGE REACHES PRODUCTION



DEPLOYMENT SUCCEEDS BUT CONTAINS OLD CODE



THE PROBLEM

- Deployment is successful.
- Application is running.
- But users see old code.
- Wrong Docker image was deployed.



WHY IT HAPPENS

- Mutable tags are used.
- The tag was overwritten.
- Caching returns old layers.
- Incorrect image referenced in deployment.



INTERVIEW FOCUS

PROVING EXACTLY WHAT WAS DEPLOYED TO PRODUCTION.

1. AVOID MUTABLE TAGS

AVOID THESE (RISKY)



latest

Changes every time.
Unpredictable.



v1

Can be overwritten.
Not reliable.



v1.2.3-3f2a1b7

Immutable.
Unique.



sha256:ab12cd...

Image digest.
Guaranteed exact match.

2. IMAGE IDENTIFICATION METHODS



IMAGE TAG

Use semantic version +
commit SHA.
Example: v1.2.3-3f2a1b7



IMAGE DIGEST

Digest never changes.
Example:
sha256:ab12cd34ef56...



COMMIT SHA

Trace image back to exact
source code.
Example: 3f2a1b7

3. ARTIFACT TRACEABILITY



CODE

Commit
3f2a1b7



BUILD

Image built
v1.2.3-3f2a1b7



REGISTRY

Pushed
@sha256:ab12cd...



DEPLOYMENT

Deployed
v1.2.3-3f2a1b7



**ALWAYS KNOW: WHAT WAS BUILT,
WHAT WAS PUSHED, WHAT WAS DEPLOYED.**

4. HOW TO PROVE WHAT WAS DEPLOYED

- 1 Check running image `kubectl get pods -o wide`
- 2 Describe pod `kubectl describe pod <pod-name>`
- 3 View image ID (digest) `kubectl get pod <pod> -o jsonpath='{.spec.containers[*].image}'`
- 4 Inspect image digest `docker inspect <image> --format '{{.RepoDigests}}'`
- 5 Check deployment history `kubectl rollout history deployment/<name>`
- 6 Compare with registry `docker pull <image>@sha256:ab12cd...`

5. BEST PRACTICES

- ✓ Never use "latest" in any environment.
- ✓ Tag images with version + commit SHA.
- ✓ Use image digests in production.
- ✓ Enable admission policies to block mutable tags.
- ✓ Keep artifact traceability in your pipeline.
- ✓ Log and record every artifact used in deployment.



**TRACEABILITY IS HOW YOU PROVE THE TRUTH.
IT PROTECTS YOUR RELEASES AND YOUR REPUTATION.**



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

DEPLOYMENT STAGE FAILS IN KUBERNETES



BUILD, TESTS AND IMAGE PUBLISHING SUCCEED, BUT DEPLOYMENT TO KUBERNETES FAILS.



INTERVIEW FOCUS

DEPLOYMENT-LAYER DIAGNOSIS

Identify why Kubernetes rejected or could not complete the deployment.



COMMON SYMPTOMS

- Deployment job fails.
- Pods are not created.
- Pods are stuck in Pending.
- ImagePull errors.
- Permission denied.
- Resources not found.



START HERE

- 1 Confirm the failed stage is the deployment stage.
- 2 Check the deployment job logs first.
- 3 Do not rerun blindly. Diagnose deliberately.
- 4 Use kubectl to get the real reason.

1. CHECK THE FUNDAMENTALS



KUBERNETES AUTHENTICATION

Can the pipeline authenticate to the cluster?

```
kubectl auth can-i --list
```



CLUSTER CONNECTIVITY

Can the pipeline reach the cluster API server?

```
kubectl cluster-info
```



RBAC PERMISSIONS

Does the service account have permission to deploy?

```
kubectl auth can-i create deploy --namespace <ns>
```



INVALID MANIFESTS

YAML syntax errors?
Unsupported fields?
Wrong API versions?

```
kubectl apply --dry-run=client -f <file.yaml>
```



NAMESPACE MISTAKES

Does the namespace exist?
Is the name correct?

```
kubectl get ns
```

2. USE KUBECTL TO FIND THE REAL CAUSE



DESCRIBE THE RESOURCE

```
kubectl describe <resource> -n <namespace>
```

Shows status, conditions, events and error reasons.



CHECK EVENTS

```
kubectl get events -n <namespace> --sort-by=.lastTimestamp
```

Events tell you what Kubernetes is trying to do.



CHECK PODS

```
kubectl get pods -n <namespace> -o wide
```

Verify Pods, status, images and node.



CHECK LOGS

```
kubectl logs <pod-name> -n <namespace> --all-containers
```

Application or init container errors.

3. COMMON ROOT CAUSES



Authentication failure (invalid kubeconfig, expired token).



Cluster unreachable (network/DNS/VPC issues).



RBAC denies the action (missing permissions).



Invalid or malformed manifests.



Wrong namespace or it does not exist.



Image pull errors (wrong image or private registry).



Quota limits exceeded (CPU, memory, objects).

4. INTERVIEWER WANTS TO SEE THIS THINKING



OBSERVE

What is the exact error?



GATHER EVIDENCE

Use logs, describe, events and metrics.



ANALYZE

Identify the failing layer and reason.



HYPOTHESIZE

Form a likely root cause.



TEST & VALIDATE

Run commands to confirm.



RESOLVE

Fix the issue safely.



VERIFY

Confirm deployment is healthy.



STRONG ENGINEERS DIAGNOSE THE DEPLOYMENT LAYER WITH EVIDENCE, NOT ASSUMPTIONS.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

DEPLOYMENT SUCCEEDS BUT APPLICATION IS BROKEN



PIPELINE IS GREEN BUT USERS REPORT FAILURES.
DEPLOYMENT SUCCESS DOES NOT EQUAL APPLICATION HEALTH.



INTERVIEW FOCUS

DEFINING REAL DEPLOYMENT SUCCESS

Validate that the application is healthy and working for users, not just deployed.



WHAT THIS SCENARIO MEANS

- The deployment completed.
- All pipeline stages passed.
- But users experience errors.
- Something is wrong in runtime.
- Your job is to prove what is actually broken and why.



EXAMPLES OF USER IMPACT

- 5xx errors
- Timeouts
- Login failures
- Missing data
- Partial functionality
- Performance degradation

1. DEPLOYMENT SUCCESS $\neq$ APPLICATION HEALTH



CODE



BUILD



TEST



SCAN



PACKAGE



DEPLOY


APPLICATION
HEALTHY


REMEMBER

A GREEN PIPELINE ONLY PROVES THAT THE SYSTEM WAS DELIVERED.

IT DOES NOT PROVE THAT USERS CAN SUCCESSFULLY USE THE APPLICATION.

2. VERIFY APPLICATION HEALTH



SMOKE TESTS

Run basic end-to-end tests that cover the most critical flows.

Example:

/health, login, create, search.



READINESS PROBES

Ensure pods are ready before receiving traffic. A failing readiness probe means the app is not actually ready.

Check:

readinessProbe, startupProbe.



APPLICATION LOGS

Logs reveal errors, exceptions, stack traces, and failed dependencies.

Check:

kubectl logs, log aggregation.



METRICS & ALERTS

Monitor error rate, latency, CPU, memory, and saturation.

Check:

Dashboards, Alerts, SLOs.



POST-DEPLOYMENT VERIFICATION

Validate that the real user experience is healthy and stable.

Check:

Synthetic tests, User actions.



3. COMMON ROOT CAUSES

- Configuration changes or missing environment variables.
- Dependency failures (DB, cache, APIs, secrets).
- Backward incompatible changes.
- Bad data or migration issues.
- External service outages or rate limits.
- Resource limits or scaling issues.
- Incorrect routing, DNS, or load balancer configuration.



4. DIAGNOSTIC APPROACH

- Confirm the user impact and scope.
- Check recent deployment timeline.
- Run smoke tests and synthetic checks.
- Inspect readiness probes and pod status.
- Review application logs and metrics.
- Identify the root cause and fix.



5. INTERVIEWER WANTS TO HEAR

- You don't stop at a green pipeline.
- You validate real user experience.
- You check probes, logs, and metrics.
- You use evidence to find the root cause.
- You verify recovery after the fix.
- You define deployment success clearly.



REAL DEPLOYMENT SUCCESS = USERS CAN RELIABLY USE THE APPLICATION EVERY TIME. VERIFY. MONITOR. IMPROVE.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

PRODUCTION DEPLOYMENT CAUSES 5xx ERRORS



**ERROR RATE SPIKES IMMEDIATELY AFTER RELEASE.
FAST, STRUCTURED RESPONSE IS CRITICAL.**



INTERVIEW FOCUS

INCIDENT RESPONSE DURING DEPLOYMENT

Prove you can quickly identify, analyze, and mitigate issues caused by a new release.



WHAT THIS MEANS

- Error rate (5xx) spikes after release.
- Users are impacted.
- The new version is likely the cause.
- You must find the root cause fast.
- Restore service with minimal risk.



USER IMPACT EXAMPLES

- 5xx errors
- Timeouts
- Slow responses
- Failed transactions
- Degraded service

1. ESTABLISH THE DEPLOYMENT TIMELINE



CODE

Commit
a1b2c3d
10:02 AM



BUILD

Build
succeeded
10:04 AM



TESTS

Tests
passed
10:07 AM



SCAN

Security scan
passed
10:09 AM



DEPLOY START

Rolling update
started
10:10 AM



DEPLOY COMPLETE

Deployment
completed
10:13 AM



The spike starts **RIGHT AFTER** the new version is fully rolled out.
Focus your investigation here.

2. QUICK TRIAGE CHECKLIST

- ✓ Confirm the 5xx spike from monitoring.
- ✓ Check which service(s) are affected.
- ✓ Check recent deployment and changes.
- ✓ Verify new version and image deployed.
- ✓ Check health checks and probes.
- ✓ Scan application logs for errors.
- ✓ Check dependencies (DB, cache, APIs).
- ✓ Compare with previous healthy version.
- ✓ Decide: mitigate, rollback, or fix forward.

3. COMPARE OLD vs NEW VERSION

COMPARE THESE

- ✓ Configuration changes
- ✓ Environment variables
- ✓ Feature flags
- ✓ Dependencies
- ✓ Database migrations
- ✓ External integrations
- ✓ Resource limits
- ✓ Security / IAM changes

WHY?

Small changes can cause big failures.

Find what is different now compared to the last known good version.



4. CHECK LOGS, METRICS & HEALTH



LOGS

Look for exceptions, stack traces, and errors.



METRICS

Error rate, latency, CPU, memory, saturation.



HEALTH CHECKS

Readiness / Liveness failing?
Upstream dependencies?



DEPENDENCIES

DB, Cache, APIs, Queues,
Third-party services.

5. ROLLBACK DECISION GUIDE



ROLLBACK IF:

- Impact is high and widespread
- Root cause is unknown
- Fix will take too long
- No safe mitigation available



DO NOT ROLLBACK IF:

- Quick fix is safe and possible
- Impact is low or isolated
- System is stable with mitigation
- Rollback risk is higher

6. INCIDENT RESPONSE FLOW



1. DETECT

Alert triggers.
Confirm impact.



2. INVESTIGATE

Gather data.
Find the cause.



3. HYPOTHEZIZE

What changed?
What broke?



4. MITIGATE

Rollback or fix
forward safely.



5. VERIFY

Error rate drops.
Users are good.



6. COMMUNICATE

Update team.
Share status.



7. POST-INCIDENT

Write timeline.
Find prevention.



**SPEED WITHOUT ACCURACY MAKES THINGS WORSE.
STAY CALM. FOLLOW THE PROCESS. RESTORE SERVICE.**



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

DATABASE MIGRATION BREAKS THE RELEASE



**NEW APPLICATION REQUIRES A SCHEMA CHANGE.
MIGRATION FAILS OR BREAKS THE PREVIOUS APPLICATION VERSION.**



INTERVIEW FOCUS

SAFE DATABASE DELIVERY

Design, troubleshoot, and explain how to safely deliver database changes without breaking the application or losing data.



COMMON SYMPTOMS

- Migration fails during deployment.
- Application cannot start.
- Previous version also stops working.
- Database schema is in an inconsistent state.
- Errors in application logs.
- Users experience downtime or errors.



KEY AREAS TO DISCUSS

- Backward-compatible migrations
- Deployment ordering
- Schema versioning
- Data validation
- Backups and recovery
- Roll-forward vs rollback
- Feature flags
- Zero or minimal downtime

1. WHY DATABASE MIGRATIONS ARE RISKY

- ⚠ Schema changes can break the application.
- ⚠ Not all changes are backward compatible.
- ⚠ Both new and old versions may be affected.
- ⚠ Data loss can occur if not planned properly.
- ⚠ Rollback can be complex or impossible.

2. CHECK BEFORE THE DEPLOYMENT

- ✓ Is the migration backward compatible?
- ✓ Have you tested the migration in a staging environment?
- ✓ Do you have a database backup?
- ✓ Have you planned the deployment order?
- ✓ Can the old application still work?
- ✓ Do you have a rollback or roll-forward plan?

3. DURING THE INCIDENT

- 1 Check deployment and migration logs.
- 2 Identify what change was applied.
- 3 Check database schema and data state.
- 4 Understand application errors.
- 5 Determine the impact on users.
- 6 Decide: rollback, fix, or roll forward.

4. MIGRATION STRATEGIES

✓ BACKWARD COMPATIBLE	New and old versions can work together.
⚙ EXPAND AND CONTRACT	Add changes first, remove later.
🔗 FEATURE FLAGS	Keep old and new logic separate.
🕒 BLUE-GREEN OR CANARY	Deploy safely with minimal impact.

5. DEPLOYMENT ORDERING

1. **DEPLOY DATABASE CHANGES**
(if backward compatible)
2. **DEPLOY APPLICATION**
(new version)
3. **VERIFY FUNCTIONALITY**
and monitor
4. **CLEAN UP**
(remove old schema if needed)

6. ROLLBACK vs ROLL FORWARD



ROLLBACK

- Return to previous version.
- Use when safe and possible.
- May require schema rollback (not always possible).



ROLL FORWARD

- Fix the issue and apply a new migration.
- Preferred when rollback is risky or not possible.
- Often safer for production.

7. IMPORTANT COMMANDS (EXAMPLES)

Check schema	\d or SHOW TABLES;
Check migration version	SELECT * FROM schema_migrations;
View recent changes	SELECT * FROM INFORMATION_SCHEMA ORDER BY timestamp DESC;
Verify data	SELECT COUNT(*) FROM table;



8. BEST PRACTICES

- ✓ Always test migrations in a staging environment.
- ✓ Use backward-compatible changes.
- ✓ Keep migrations small and focused.
- ✓ Have a tested backup and recovery plan.
- ✓ Document the migration process.
- ✓ Monitor database and application after deployment.
- ✓ Plan for both rollback and roll forward.



9. INTERVIEWER WANTS TO HEAR

- ✓ How you assess the risk.
- ✓ How you test the migration.
- ✓ How you plan the deployment order.
- ✓ How you handle failures.
- ✓ How you decide between rollback or roll forward.
- ✓ How you ensure data safety and minimal downtime.



**A SUCCESSFUL RELEASE IS NOT JUST ABOUT DEPLOYING CODE.
IT IS ABOUT DELIVERING A WORKING, STABLE, AND SAFE SYSTEM.**



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

TWO PIPELINES DEPLOY TO PRODUCTION AT THE SAME TIME



CONCURRENT RELEASES INTERFERE WITH EACH OTHER



THE PROBLEM

- Two (or more) pipelines deploy to production simultaneously.
- Releases overwrite each other.
- Users get inconsistent versions.
- Configurations and databases may be updated in the wrong order.
- Hard to know which version is actually running.



WHY IT HAPPENS

- No deployment locks or controls.
- Multiple teams deploy at once.
- Manual deployments outside CI/CD.
- Long running deployments.
- No environment protection rules.
- Lack of release coordination.



INTERVIEW FOCUS

PIPELINE SAFETY THROUGH CONCURRENCY CONTROLS AND RELEASE COORDINATION.

1. WHAT CAN GO WRONG?



2. KEY SAFETY CONTROLS



DEPLOYMENT LOCKS

Ensure only one deployment to production runs at a time.



CONCURRENCY CONTROLS

Limit the number of concurrent deployments using CI/CD features.



ENVIRONMENT PROTECTION

Require approvals and checks before releasing to production.



RELEASE COORDINATION

Teams communicate and schedule releases.



TIMEOUTS AND ROLLBACKS

Set timeouts and auto-rollback on failures or conflicts.

3. EXAMPLES IN POPULAR PLATFORMS

PLATFORM	HOW TO CONTROL CONCURRENCY
 GITHUB ACTIONS	- Use environment protection rules - Require reviewers - Use concurrency: group + cancel-in-progress
 GITLAB CI/CD	- Use "resource group" - Use environment locks - Define deployment approvals
 JENKINS	- Use "Lockable Resources" plugin - Use throttling - Use milestones to prevent old builds
 AZURE PIPELINES	- Use environment checks and approvals - Use exclusive lock (checks) - Limit parallel jobs and stages

4. GOOD PRACTICES

- Always protect production environments.
- Never allow parallel deployments to production.
- Use locks, approvals and health checks.
- Keep deployments short and reliable.
- Monitor deployments and failures.
- Communicate with teams before releasing.

5. INTERVIEW ANSWER FRAMEWORK



SAFE PIPELINES. PREDICTABLE RELEASES. HAPPY USERS.

PIPELINE IS TAKING 45 MINUTES



IT WORKS, BUT FEEDBACK IS TOO SLOW



THE PROBLEM



- Pipeline takes 45 minutes.
- It eventually succeeds.
- Developers wait too long for feedback.
- Slow feedback reduces productivity.
- We need speed without sacrificing quality.



WHY IT MATTERS

- ✓ Slow feedback delays fixes.
- ✓ Context switching increases.
- ✓ Developers bypass quality gates.
- ✓ Releases take longer.
- ✓ Team velocity decreases.
- ✓ Happy developers need fast and reliable pipelines.



INTERVIEW FOCUS

**PERFORMANCE
WITH RELIABILITY.**

**OPTIMIZE SMARTLY.
NEVER REMOVE
ESSENTIAL TESTS.**

1. FIND THE SLOWEST STAGE FIRST



DON'T GUESS. MEASURE.

- Look at the pipeline duration breakdown.
- Identify the stage that takes the longest.
- Focus your optimization there first.

EXAMPLE PIPELINE TIMELINE (TOTAL: 45 MIN)

CODE



3 MIN

BUILD



12 MIN

TEST



20 MIN

SCAN



5 MIN

PACKAGE



3 MIN

DEPLOY



2 MIN



TEST STAGE IS THE BOTTLENECK (20 MIN)

2. OPTIMIZATION STRATEGIES THAT WORK

CACHING



- Cache dependencies (pkg, npm, pip, maven).
- Cache build artifacts.
- Cache test dependencies.
- Use remote cache when possible.

PARALLEL JOBS



- Run independent jobs in parallel.
- Split tests by files, modules or tags.
- Use matrix builds smartly.
- Reduce total wall time.

DEPENDENCY INSTALLATION



- Dependencies are often the biggest slowdown.
- Use lock files.
- Install only what is needed.
- Pre-bake dependencies in Docker images when appropriate.

DOCKER LAYER CACHING



- Order Dockerfile instructions from least to most frequently changing.
- Leverage layer cache.
- Avoid invalidating cache unnecessarily.
- Use BuildKit and multi-stage builds.

RUNNER CAPACITY



- More runners = more parallelism.
- Use appropriate runner size.
- Monitor CPU, memory, disk, and network.
- Autoscale runners when needed.

OTHER TIPS



- Skip unnecessary steps in PRs.
- Use incremental analysis.
- Limit log verbosity.
- Keep artifacts small and necessary.

3. WHAT NOT TO DO

- ✗ Don't remove unit or integration tests.
- ✗ Don't reduce security scan coverage.
- ✗ Don't skip quality gates.
- ✗ Don't deploy without proper checks.
- ✗ Don't optimize blindly. Measure and verify.



4. INTERVIEW ANSWER FRAMEWORK



MEASURE
Check pipeline analytics. Find slowest stage.



ANALYZE
Understand why it's slow. Gather evidence.



OPTIMIZE
Apply the right optimization strategies.



VALIDATE
Ensure quality is not affected. Rerun and verify.



MONITOR
Track metrics and continue improving.



**FAST PIPELINES EMPOWER TEAMS.
NEVER TRADE QUALITY FOR SPEED. OPTIMIZE WITH INTELLIGENCE.**



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

EMERGENCY ROLLBACK DURING A FAILED DEPLOYMENT

**NEW RELEASE IS DAMAGING PRODUCTION**

THE SITUATION



- New release causes errors, outages, or performance issues.
- Users are impacted.
- Monitoring shows degradation.
- Immediate action is required.



WHEN SHOULD YOU ROLLBACK?

- ✓ High error rate impacting users.
- ✓ Critical functionality is broken.
- ✓ Performance severely degraded.
- ✓ No quick, safe fix available.
- ✓ Issue is confirmed to be release-related.

**ROLLBACK FAST,
BUT ROLLBACK SAFE.**

INTERVIEW FOCUS

**RESTORING SERVICE
SAFELY IS MORE
IMPORTANT THAN
DEPLOYING FAST.**

**BE CALM.
BE METHODICAL.
FOLLOW THE PROCESS.**

1. ROLLBACK STRATEGY OVERVIEW



2. ROLLBACK METHODS (KUBERNETES)

- ✓ `kubectl rollout undo deployment/<name>`
- ✓ Use rollout history to find previous revision.
- ✓ Rollback to a specific revision number.
- ✓ Verify all Pods are running the old version.
- ✓ Check Services, ConfigMaps, and Ingress.

USEFUL COMMANDS

```
kubectl rollout history deployment/<name>
kubectl rollout undo deployment/<name>
kubectl rollout undo deployment/<name> --to-revision=3
kubectl get pods -l app=<name>
kubectl describe deployment/<name>
```



3. KEY CONSIDERATIONS



KNOWN-GOOD ARTIFACT
Rollback to a proven, stable version.



KUBERNETES HISTORY
Use rollout history and revisions.



DATABASE COMPATIBILITY
Ensure schema is backward compatible.



FEATURE FLAGS
Disable the new feature if possible.



DEPENDENCIES
Check external services and configs.



4. DATABASE COMPATIBILITY CHECKLIST

- ✓ Is the new version compatible with the existing schema?
- ✓ Did the release run any migrations?
- ✓ Are migrations backward compatible?
- ✓ Can the old application version run with the current database state?
- ✓ Do NOT rollback if it will break data integrity.



5. FEATURE FLAGS TO THE RESCUE

- ✓ Turn off the new feature immediately.
- ✓ Reduce blast radius without rolling back.
- ✓ Gives time to diagnose the issue.
- ✓ Re-enable only after the fix is verified.



6. VERIFICATION AFTER ROLLBACK

- ✓ Confirm all Pods are healthy.
- ✓ Check application health endpoints.
- ✓ Monitor error rate and latency.
- ✓ Validate critical user workflows.
- ✓ Ensure background jobs are running.
- ✓ Watch logs and metrics for stability.

7. INTERVIEW ANSWER FRAMEWORK

**ASSESS**

Understand impact and root cause.

**DECIDE**

Decide if rollback is the right move.

**EXECUTE**

Rollback using a safe method.

**VERIFY**

Verify recovery and system stability.

**DOCUMENT**

Document the incident and lessons.



**A GOOD ENGINEER DOESN'T JUST DEPLOY.
A GREAT ENGINEER RESTORES SERVICE SAFELY.
PROTECT USERS. PROTECT DATA. LEARN AND IMPROVE.**



veriqta



veriqta



veriqta



veriqta



veriqta



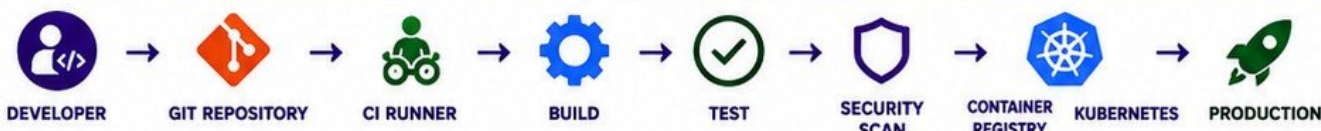
veriqta

FINAL CI/CD PRODUCTION INTERVIEW SIMULATION



YOUR GOAL: RESTORE SERVICE SAFELY AND PREVENT RECURRENCE

1. DELIVERY PATH (END-TO-END)



2. SYMPTOMS AFTER DEPLOYMENT

- | | |
|--|---|
| <ul style="list-style-type: none">⚠ Pipeline worked yesterday.⚠ Build succeeds.⚠ Tests succeed.⚠ Security scan succeeds.⚠ Image push succeeds.⚠ Deployment reports success. | <ul style="list-style-type: none">⚠ New Pods start CrashLoopBackOff.⚠ Some users receive HTTP 502.⚠ Error rate rises after the deployment.⚠ One environment variable is missing.⚠ Previous release was healthy. |
|--|---|

3. WHAT YOU MUST DO (STEP-BY-STEP)

- | | |
|--|---|
| <ol style="list-style-type: none">1 Establish production impact.2 Identify the first failing layer.3 Build a deployment timeline.4 Inspect pipeline logs.5 Inspect Kubernetes events.6 Inspect application logs.7 Compare old and new configuration. | <ol style="list-style-type: none">8 Form and test a hypothesis.9 Identify the likely root cause.10 Decide whether to rollback.11 Restore service safely.12 Verify recovery.13 Explain how to prevent recurrence. |
|--|---|

4. INVESTIGATION CHECKLIST

- | | | | | |
|--|--|---|--|---|
| 1. ESTABLISH IMPACT <ul style="list-style-type: none">✓ Check error rate (5xx).✓ Identify affected users.✓ Check latency & response time.✓ Verify business impact.✓ Communicate status. | 2. TIMELINE <ul style="list-style-type: none">✓ Note deployment time.✓ When did errors start?✓ What changed?✓ Compare with previous healthy release. | 3. LOGS & EVENTS <p>CHECK THESE</p> <ul style="list-style-type: none">✓ CI/CD pipeline logs✓ Kubernetes events (kubectl get events)✓ Pod logs (kubectl logs)✓ Ingress/Load Balancer logs✓ Application logs✓ Monitoring/Alerts | 4. CONFIG COMPARISON <ul style="list-style-type: none">✓ Compare environment variables.✓ Compare ConfigMaps & Secrets.✓ Compare Helm values or manifests.✓ Check changes in application config.✓ Verify database connection settings. | 5. HYPOTHESIS & ROOT CAUSE <ul style="list-style-type: none">✓ Form a clear hypothesis.✓ Test the hypothesis.✓ Identify the likely root cause. <div>LIKELY ROOT CAUSE EXAMPLE<p>Missing environment variable required by the application.</p></div> |
|--|--|---|--|---|

6. DECIDE & RESTORE

- | | |
|--|---|
| ROLLBACK IF: <ul style="list-style-type: none">✓ Impact is high.✓ Root cause is not quick to fix.✓ Data risk exists.✓ No safe forward fix available. | ROLLBACK USING: <ul style="list-style-type: none">✓ Known-good image tag or digest.✓ Kubernetes rollout undo.✓ Helm rollback.✓ Previous stable configuration. |
|--|---|

7. VERIFY RECOVERY

- ✓ Pods are Running & Ready.
- ✓ Error rate is back to normal.
- ✓ Health checks are passing.
- ✓ Users can access the service.
- ✓ Monitor for stability.

8. PREVENT RECURRENCE

- ✓ Add environment variable validation at startup.
- ✓ Add smoke tests for critical flows.
- ✓ Use IaC to manage all configuration.
- ✓ Add automated config checks.
- ✓ Improve monitoring & alerting.
- ✓ Document the incident & lessons.

KEY COMMANDS

```
kubectl get pods -n <ns>
kubectl describe pod <pod> -n <ns>
kubectl logs <pod> -n <ns> --tail=200
kubectl get events -n <ns> --sort-by=.metadata.creationTimestamp
kubectl rollout history deployment/<name> -n <ns>
kubectl rollout undo deployment/<name> -n <ns>
kubectl get configmap,secret -n <ns>
```



**A GREAT ENGINEER ISN'T THE ONE WHO NEVER FAILS.
A GREAT ENGINEER IS THE ONE WHO RESTORES SERVICE
SAFELY, LEARNS FAST, AND PREVENTS IT NEXT TIME.**



**INTERVIEW FOCUS: CAN YOU THINK LIKE AN ENGINEER
AND ACT LIKE A LEADER DURING AN INCIDENT?**



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta